beeDB - A Distributed Database System

47857 KETCY JENIFER

DEPARTAMENTO **CIÊNCIA E TECNOLOGIA**

INFORMATICS ENGINEERING
Curricular Unit: Distributed Systems
Name of the teacher: Joao Macedo

Contents

Project Report: beeDB - A Distributed Database System

Abstract

beeDB is a lightweight distributed database system implemented using Node.js. It mimics the functionality of a distributed data store by utilizing multiple data nodes and a master controller for query forwarding and storage management. The system uses the Raft consensus algorithm to elect a leader among the nodes and applies the Two-Phase Commit (2PC) protocol for distributed transactions. This report outlines the architecture, key modules, implementation strategy, and testing procedures of beeDB, designed for educational and experimental purposes in distributed systems.

1. Introduction

Distributed systems are at the core of modern software infrastructure. From cloud databases to scalable microservices, understanding how distributed systems function is critical. beeDB is an attempt to simulate a simplified distributed database to gain practical insights into distributed data management, replication, node coordination, leader election, and distributed transaction logging.

2. Problem Statement

Managing data across multiple nodes while maintaining consistency and reliability is a fundamental challenge in distributed systems.
beeDB aims to address:

- Query forwarding based on data sharding logic
- Coordinated write and read operations
- Distributed logging and error handling
- Configurable node management
- Leader election using the Raft algorithm for fault-tolerant control
- Data consistency using Two-Phase Commit (2PC) protocol

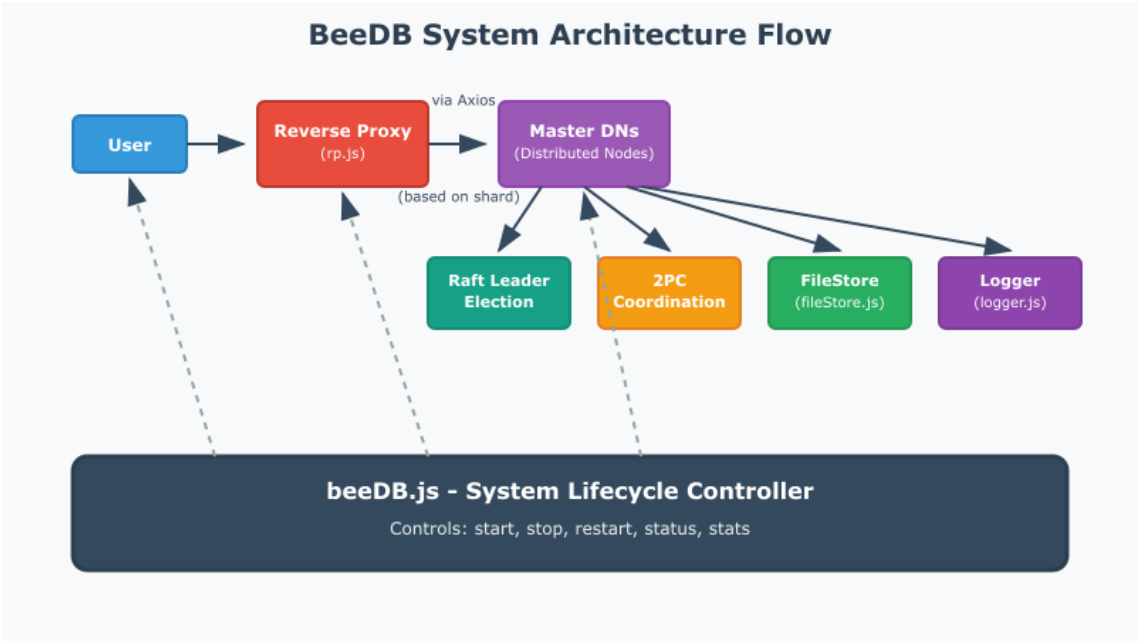
3. System Design

Architecture Overview

- **Master Node (beeDB.js)**: Controls cluster operations (start, stop, restart, stats) and launches all nodes.
- **Reverse Proxy (rp.js)**: Accepts client requests, handles sharding and forwards them to appropriate DN masters.
- **Data Nodes (dn/server.js)**: Handle CRUD operations using local file storage, participate in Raft election, and coordinate via 2PC.
- **Raft Module (utils/raft.js)**: Coordinates leader election among nodes.
- **2PC Module (utils/twophase.js)**: Ensures atomic commits across DNs.
- **Configuration Layer**: JSON files define server details and relationships.
- **File Storage (myFS/fileStore.js)**: Handles disk operations.
- **Logger (logWrapper/logger.js)**: Provides centralized logging per operation.
- **Web Framework**: Express is used to handle HTTP routing.

- **HTTP Client**: Axios is used for inter-node communication.

System architecture



4. Technology Stack

Language: JavaScript (Node.js)
Framework: Express.js for HTTP server
HTTP Client: Axios
Configuration: JSON
Storage: File-based (local)
Architecture: Sharded Node Design with Raft consensus and 2PC transaction consistency
Monitoring Tools: forever-monitor, PowerShell commands for process control
Other Tools: npm for package management

5. Implementation Details

- **beeDB.js**: Starts RP with forever
- Central cluster controller which supports commands like start, stop, restart, status, stats. It launches all DN nodes and RP using spawn and forever-monitor.
- **rp.js**: Routes CRUD requests to correct DN master based on sharded MD5 hash. Handles /set_master updates from Raft leaders. Uses express-http-proxy. Shards using MD5. Logs "I'm here" before each forward. Accepts /set_master updates from Raft DNs.
- **dn/server.js**: Each data node runs independently, participates in Raft elections, and processes CRUD using the 2PC protocol.
- **Raft Module**: Handles /raft/vote, /raft/heartbeat, and leader election. eader election via term-based voting and heartbeats. Leader notifies RP via
- **2PC Module**: Coordinates /internal/prepare, /internal/commit, /internal/abort operations across peers.
- **Logger**: Winston logger wrapped with init, log, setLogLevel, end. Logs have error codes like 60001 and are used across all modules.prv: `/admin/loglevel` is local-only

- **Stats API**: Each DN exposes /stats, and beeDB.js aggregates all stats using Axios.

6. Features

- Configurable multi-node setup
- Sharding using MD5(key)
- Local node storage with file system simulation

- HTTP-based routing and inter-node communication
- Raft-based leader election for fault tolerance
- Two-Phase Commit (2PC) protocol for atomic write consistency
- Central cluster controller with lifecycle commands
- Logging and stats monitoring per node
- CLI lifecycle control (start/stop/status)
- IP-based route protection

7. Testing and Evaluation

Keys tested:

- Number: 565656
- String: "me@upt.pt"
- Object: { "a": 7878, "b": "bbb", "me": "U" }

Validated:

- Correct DN routing
- Replication to all DNs
- Raft leader election and failover
- CRUD reflected in /stats API
- RP aggregates DN stats using Axios

- Testing was performed by running various configurations and verifying:
- Correct routing of queries based on hash sharding
- Data creation and read consistency via 2PC
- Raft leader election and re-election under failure scenarios
- Proper functioning of the reverse proxy and RP→DN forwarding
- Logging accuracy across modules
- Process lifecycle management using beeDB.js

8. Conclusion

beeDB offers a compact, hands-on approach to understanding the core principles of distributed systems. With integrated Raft leader election, Two-Phase Commit for consistency, and a modular control structure, it simulates several real-world concepts in distributed databases. The system is educational, portable, and practical for academic and experimental purposes.

9. How to Run beeDB

9.1. Prerequisites

Node.js (v14 or higher)
npm installed
Windows PowerShell (or compatible terminal)
Internet access (for downloading dependencies)

9.2. Install Dependencies

npm install

9.3. Start the beeDB Cluster

node beeDB.js start

9.4. Available Commands

Command	Description
node beeDB.js start	Starts all Data Nodes and Reverse

Command	Description
	Proxy
node beeDB.js stop	Stops all processes
node beeDB.js restart	Restarts the full cluster
node beeDB.js status	Displays status of all nodes
node beeDB.js stats	Shows CRUD operation stats from all DNs

9.5. API Endpoints

Reverse Proxy

POST /db/c: Create key-value
GET /db/r?key=...: Read key
POST /db/u: Update key-value
GET /db/d?key=...: Delete key

Internal & Admin:

- /set_master: Raft → RP (DNp)
- /election, /maintenance: DN internal use (DNp)
- /admin/loglevel: Change log level (prv)
- /status, /stats: Node health & CRUD count
- /stop: Graceful shutdown (RPt only)

9.6. Example Usage (with curl)

Create key-value

curl -X POST http://localhost:8000/db/c -H "Content-Type: application/json" -d '{"key": "user1", "value": {"name": "Alice"}}'

Read value

curl "http://localhost:8000/db/r?key=user1"

10. Project Flow

Startup Phase:
beeDB.js start launches all Data Nodes (DNs) and the Reverse Proxy (RP).
Each DN registers itself and initializes Raft.
Leader Election (Raft):
All DNs start in follower mode.
Raft leader is elected based on timeouts and votes.
Leader notifies RP of its master status.
Client Interaction:
Clients send requests (CRUD) to the RP via /db/c, /db/r, etc.
RP uses MD5-based sharding to determine which DN handles the key.
2PC Protocol:
For create, update, and delete requests:
RP forwards request to the master DN.
Master initiates prepare phase with its peers.
If all peers respond "ready", the commit phase proceeds.
Data Persistence:
Values are stored as JSON files in the file system.
Reads are served directly by the master DN.

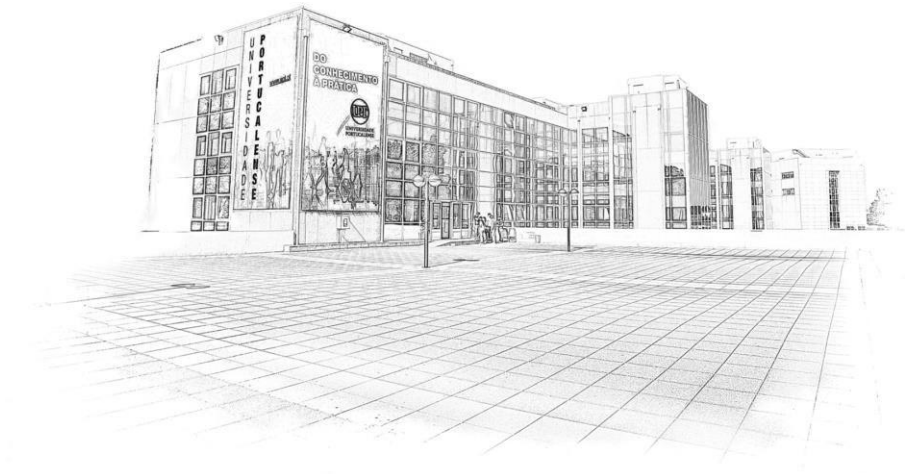
Cluster Monitoring:
Status and stats are accessible via /status and /stats APIs.

Logs are maintained using the centralized logger module.

Shutdown or Restart:

beeDB.js stop terminates all services.

beeDB.js restart first stops and then restarts all services.



Universidade Portucalense Infante D. Henrique | Rua Dr. António Bernardino de Almeida, 541
4200-072 Porto | Telefone: +351 225 572 000 | email: upt@upt.pt